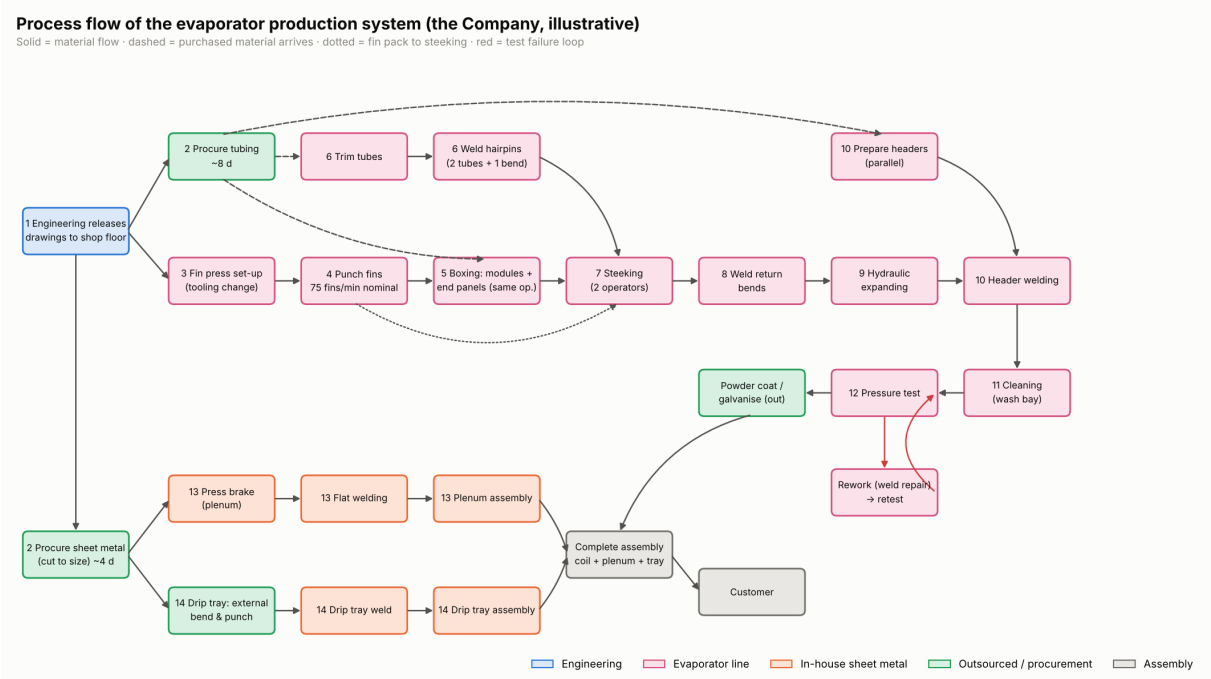
Evaporator VSM Simulation

User Guide: what the project does, how to set it up, how to run it, and how to read every result



The evaporator production system modelled in this project (colour-coded by department).

> **Illustrative and anonymised data.** The organisation is called "the Company". All times, probabilities and lead times are illustrative inputs used to demonstrate the method. They are not measured company data.

Repository: https://github.com/fadeelphillips29-cmd/evaporator-vsm-simulation

Every number quoted in this guide was read from the committed result files in `results/` when the guide was generated.

Contents

1. What this project is and what it aims to achieve 4

1.1 The questions the study answers . . . 4

1.2 The answers in one page . . . 4

2. The production system being modelled 5

3. How the model works (plain language) 5

3.1 Discrete-event simulation (DES) . . . 5

3.2 Monte Carlo replications . . . 6

3.3 Common random numbers (fair comparisons) . . . 6

3.4 Two-level Monte Carlo (uncertainty about inputs) . . . 6

3.5 Verification and validation . . . 6

4. Navigating the repository 6

5. Setting up your computer (from zero) 7

5.1 What a terminal is . . . 7

5.2 Install Python (version 3.11 or newer) . . . 7

5.3 Install Git (to download the project) . . . 8

5.4 Download the project . . . 8

5.5 Create and activate a virtual environment . . . 8

5.6 Install the project's libraries . . . 8

5.7 Check that everything works . . . 8

5.8 Optional: make, VS Code, Jupyter . . . 8

6. Running the simulation, step by step 9

Step 1 - Hand calculation (the whiteboard VSM) . . . 9

Step 2 - Analytical equations (fin press, weld stations, Kingman) . . . 9

Step 3 - Input analysis (which probability distribution?) . . . 10

Step 4 - Baseline Monte Carlo (the current state) . . . 10

Step 5 - Validation and calibration . . . 11

Step 6 - Input uncertainty (two-level Monte Carlo) . . . 11

Step 7 - Scenarios S0-S7 . . . 12

Step 8 - Value stream maps . . . 13

Step 9 - Demand sweep (the hockey stick) . . . 14

Step 10 - Sensitivity (tornado chart) . . . 14

Step 11 - Update the documentation tables . . . 15

Step 12 - The analysis notebook 15
Step 13 - The interactive dashboard 15

7. What every metric means **16**

8. Editing the model parameters **17**

8.1 Where and how 17
8.2 The parameters you are most likely to change, and why 17
8.3 Worked example: you measured steeking at 7.6 hours 18
8.4 Defining a new scenario 18
8.5 Replacing the illustrative data with real data 18

9. Using the dashboard **18**

10. Troubleshooting **19**

11. Quick reference card **19**

1. What this project is and what it aims to achieve

The Company builds evaporators (called **EVA** in the model) in work orders of three. An evaporator is the finned coil inside a refrigeration or air-conditioning unit. Each one passes through about 15 operations, some run by the Company and some by outside suppliers. The Company drew a **value stream map (VSM)** on a whiteboard. That hand calculation suggested that **steeking** (two operators lacing tube circuits through the stack of fins) limits how many evaporators can be built per week.

A whiteboard VSM uses average times and treats every step as if it never varies. Real production does vary: a weld takes longer than usual, a machine stops, tubing arrives late, three units queue behind each other, and two branches must both finish before an assembly can start. Those interactions decide the real lead time, and averages cannot show them. This project builds a computer simulation that includes them.

1.1 The questions the study answers

- **Does the simulation reproduce the whiteboard VSM?** With all randomness switched off it must give exactly the hand-calculated numbers. This proves the model is built correctly.
- **Which resource is the bottleneck, how sure are we, and does it move?** If you fix steeking, does header welding become the new constraint?
- **Which improvement is worth doing?** The candidates are a 3rd steeking operator, steeking standard work, a 2nd header welder, better fin-press OEE, one-piece flow, and CONWIP release control.
- **At what demand does lead time explode?** This is the "hockey stick" curve.
- **How much does it matter that the steeking time is only an estimate?** Stakeholders guessed anything from 6 to 20 hours per evaporator.

1.2 The answers in one page

Question	Answer from the committed results
Current performance (3 EVA/week)	Throughput 2.99 EVA/week; order lead time 23.1 working days on average (95th percentile 28.2); 97.2 % of orders delivered within the 6-week promise.
Bottleneck	Steeking, in 100 % of simulated years (about 52 % busy at current demand). Header welding is second (about 48 %).
Where lead time explodes	Lead time is almost flat up to 5 EVA/week (24.7 d). It rises at 5.5 (26.4 d) and explodes at 5.75-6 EVA/week (32.4 d at 6). Steeking capacity is 5.71 EVA/week.
Best single improvement	One-piece flow (S5): -4.48 days (-19.4 %) at current demand, with no capital.
Recommended package (S7)	-5.24 d (-22.7 %) at 3 EVA/week and -6.28 d (-25.4 %) at 5 EVA/week. It stays stable at 6 EVA/week (lead time 19.0 d).
Bottleneck shift	Adding a 3rd steeker (S1) moves the bottleneck to header welding in 100 % of simulated years.
A trap to avoid	Raising fin-press OEE from 70 % to 85 % (S4) changes lead time by exactly zero, because the fin press always finishes days before tubing arrives.
Biggest uncertainty	Not knowing the true steeking time explains 86 % of the lead-time variance at 5 EVA/week. The 16-20 h guesses would make even today's demand impossible to meet.

The rest of this guide explains how these answers are produced, so you can reproduce them, check them and change them.

2. The production system being modelled

Read the flow in figure 1 (page 1) from left to right. Each box colour is a department: **blue** = engineering, **pink** = evaporator line, **orange** = in-house sheet metal, **green** = outsourced or purchased, **grey** = assembly. The same colours are used in every diagram.

Step	What happens	Why it matters in the model
1 Engineering release	Drawings released to the shop floor (about 1 working day).	A delay at the start of every order.
2 Procurement	Sheet metal (about 4 d) and tubing (about 8.75 d) are ordered.	Tubing is the longest item on the critical path. No tube work can start until it arrives.
3-4 Fin press	Setup (30 min), then punch 4,000 fins per EVA at 75 fins/min nominal, about 70 % OEE.	Mechanistic model: scrap, stops, reel changes.
5 Boxing	The fin-press operator also sets up the modules.	A **shared operator**: one person, two jobs.
6 Trim tubes + weld hairpins	54 welds of 4 min = 216 min per coil.	Produces the circuits needed for steeking.
7 Steeking	2 operators, about 420 min (7 h) per EVA.	**The bottleneck.** The time is only an estimate.
8-9 Return bends, expansion	144 min and 87 min per coil.	Sequential coil work.
10 Header welding	48 welds of 8 min = 384 min (6.4 h) per coil. Headers are prepared in parallel.	**The near-bottleneck.** It also repairs test failures.
11-12 Wash, pressure test	45 min and 90 min. 4 % of coils fail and are reworked.	Rework loop back to the header welder.
Coating	Outsourced, about 3 days.	A calendar delay.
13-14 Plenum, drip tray	Sheet-metal parts. The drip tray is bent externally (about 5 days).	Parallel branches that must be ready at assembly.
Complete assembly	2 operators, 150 min. Joins coil + plenum + drip tray.	A **join**: it waits for the slowest of the three branches.

Key terms. A **work order** is 3 EVA released together. The **transfer batch** is how many EVA move to the next station together: 3 in the baseline, 1 for one-piece flow. **Takt time** is the available time divided by demand: 2,400 working minutes per week / 3 EVA = 800 min per EVA. A **bottleneck** is the resource with the highest load, which limits throughput.

3. How the model works (plain language)

3.1 Discrete-event simulation (DES)

A DES model is a computer "replay" of the factory. Orders arrive. Each operation asks for its operator or machine, and waits in a queue if that resource is busy. It holds the resource for a time drawn from a probability distribution, then hands the evaporator on. The program jumps from one event (start, finish, breakdown, arrival) to the next, so it can simulate a full working year in about 0.1 seconds. The model uses the Python library **SimPy**.

3.2 Monte Carlo replications

One simulated year is one random story. To make reliable statements, the model is run many times with different random numbers. Each run is a **replication**: 300 for the main results and 30 in quick mode. Every reported value is the average over the replications, with a **95 % confidence interval (CI)**. The CI is the range in which the true long-run average lies with 95 % confidence. A narrow CI means a precise estimate.

3.3 Common random numbers (fair comparisons)

When two scenarios are compared, replication number 17 in each scenario sees exactly the same orders, the same operation times and the same test failures. Any difference therefore comes from the change you made, not from luck. Differences are reported as **paired differences** with their own 95 % CI. If that CI does not include 0, the improvement is statistically significant.

3.4 Two-level Monte Carlo (uncertainty about inputs)

Process variability ("this weld took 9 minutes instead of 8") is different from not knowing an input ("is steeking really 7 h, or 10 h?"). The two-level analysis draws a plausible steeking time from 360-600 min/EVA for each replication, and then splits the total variation into the part caused by not knowing the input and the part caused by everyday variability.

3.5 Verification and validation

- **Verification** asks whether the model is built right. With all randomness off, the simulation must equal the hand calculation exactly. It does, for all 19 checked quantities.
- **Validation** asks whether it is the right model. It is compared with 26 weeks of "observed" data. The target is an error of 5 % or less on lead time.

4. Navigating the repository

After you download the project (section 5) you get the folder structure below. You only **need** to touch the two files in `config/` to change the model. Everything else is code, outputs or documentation.

Path	What it contains	When you use it
config/line_parameters.yaml	**Every input**: times, crews, demand, lead times, batch sizes, run length.	To change the factory or the experiment.
config/scenarios.yaml	The improvement scenarios S0-S7 as changes to the baseline.	To define a new what-if scenario.
data/time_study_samples.csv	40 synthetic stopwatch readings per operation.	Replace with real time-study data.
data/observed_weekly_output.csv	26 weeks of "measured" output and lead times.	Replace with real production records for validation.
src/evapsim/	The Python package (the model itself), described below.	Only to change the model logic.
app/dashboard.py	Interactive web dashboard (Streamlit).	Point-and-click what-if runs.
notebooks/analysis.ipynb	A guided Jupyter walkthrough of the results.	To read the study as a story.
results/*.csv, results/figures/*.png	All output tables and charts.	To read results.
docs/	SOP, assumptions register, model description, build log, VSM diagrams.	Reference.
tests/	48 automated checks.	Run them after every change.
Makefile	Short commands (make baseline, make all, ...).	Running the pipeline.

Path	What it contains	When you use it
README.md	Project front page with the key results.	Overview.

Inside src/evapsim/ (the model code)

File	Role
config.py	Reads and checks the YAML. Applies time scaling and crew effects.
equations.py	Hand-calculation formulas: fin-press OEE, weld metrics, bottleneck, Kingman, critical path.
model.py / resources.py	The SimPy simulation: orders, operations, queues, joins, rework, CONWIP.
runner.py / metrics.py	Runs many replications in parallel and computes KPIs and CIs.
scenarios.py / sensitivity.py	Scenario comparison, demand sweep, tornado chart.
validation.py	Comparison with observed data, calibration, Kingman check.
vsm.py / plots.py	Value stream maps and all charts.
pipeline.py / __main__.py	The command-line steps (python -m evapsim ...).
report.py	Writes result tables into README and docs automatically.

5. Setting up your computer (from zero)

This section assumes you have never installed Python or used a terminal. You need to do it only once per computer. Allow 20-30 minutes.

5.1 What a terminal is

A **terminal** (also called the command line) is a window where you type commands instead of clicking. Every grey box in this guide is something you type into it, pressing **Enter** after each line.

Operating system	How to open a terminal
Windows 10/11	Press the Windows key, type **PowerShell**, click Windows PowerShell.
macOS	Press Cmd + Space, type **Terminal**, press Enter.
Linux	Press Ctrl + Alt + T, or open Terminal from the applications menu.

5.2 Install Python (version 3.11 or newer)

1. Go to **https://www.python.org/downloads/** and download the latest Python 3 (3.11, 3.12 or 3.13).
2. **Windows:** run the installer. **Tick the box "Add python.exe to PATH"** on the first screen, then click Install Now. If you forget this box, the commands below will not be found.
3. **macOS:** open the downloaded .pkg and follow the steps. **Linux:** Python is usually installed already. If not, run `sudo apt install python3 python3-venv python3-pip` (Ubuntu/Debian).
4. Close and reopen the terminal, then check the installation:

```
python --version        # Windows
python3 --version       # macOS / Linux
# Expected: Python 3.11.x or higher
```

> On macOS and Linux the command is usually `python3`, and on Windows it is `python`. After you activate the virtual environment (5.5), `python` works everywhere.

5.3 Install Git (to download the project)

1. Download Git from **https://git-scm.com/downloads** and install it with the default options. macOS may offer to install the "command line developer tools": accept.
2. Check it in a new terminal: `git --version`.

No Git? On the GitHub page, click the green **Code** button, then **Download ZIP**, and unzip it. Then skip to 5.5 and open the terminal in the unzipped folder.

5.4 Download the project

```
cd Documents
git clone https://github.com/fadeelphillips29-cmd/evaporator-vsm-simulation.git
cd evaporator-vsm-simulation
```

cd means "change directory", that is, go into a folder. The terminal is now "inside" the project. Every later command must be run from this folder.

5.5 Create and activate a virtual environment

A **virtual environment** is a private Python installation for this project. Its libraries do not interfere with anything else on your computer. Create it once:

```
python -m venv .venv        # Windows
python3 -m venv .venv       # macOS / Linux
```

Activate it. You must do this every time you open a new terminal to work on the project:

```
.venv\Scripts\Activate.ps1      # Windows PowerShell
.venv\Scripts\activate.bat      # Windows Command Prompt
source .venv/bin/activate        # macOS / Linux
```

When it is active, the prompt starts with `(.venv)`.

> **Windows error "running scripts is disabled on this system"?** Run this once, then activate again: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (answer Y).

5.6 Install the project's libraries

```
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
```

This downloads SimPy, NumPy, pandas, SciPy, matplotlib, Streamlit and the other libraries listed in `requirements.txt`. The last line installs the model itself (`evapsim`) so its commands work from the project folder. It takes 2-5 minutes.

5.7 Check that everything works

```
python -m pytest
# Expected last line:  48 passed in ...s
```

The 48 automated tests check the equations, the exact match with the hand calculation, Little's law, the join logic and more. If they pass, your setup is correct.

5.8 Optional: make, VS Code, Jupyter

- **make** provides the short commands (`make baseline`). It is built into macOS and Linux. On Windows you can skip it and use the `python -m evapsim` commands instead, which do exactly the same thing. Section 6 gives both.

- **Visual Studio Code** (https://code.visualstudio.com) is a free editor that makes editing the YAML file and browsing results easier. Open the project folder with File, Open Folder. It has a built-in terminal (Terminal, New Terminal).
- **Jupyter** is installed with the requirements and is used for the notebook (step 12).

6. Running the simulation, step by step

The steps below follow the order of the study. Each step says **what to type**, **what it does**, **how long it takes**, **which files it creates** and **what the results mean**. Run them from the project folder with the virtual environment active. Runtimes are for a 4-core laptop.

> **Shortcuts.** `make all` runs everything (about 16 minutes). `make quick` runs everything with 30 replications into `results/quick/` (about 5 minutes) and is good for a first try. Add `--quick` to any `python -m evapsim` command for the same effect. The committed results already exist, so you can also skip straight to reading them.

Step 1 - Hand calculation (the whiteboard VSM)

```
python -m evapsim handcalc
```

What it does. It recomputes the whiteboard VSM from the inputs, without any simulation. For each station it calculates minutes per EVA, minutes per 3-EVA order, operators, labour content, capacity per shift and per week, and utilisation at demand. It then finds the critical path (the longest chain of steps) for one order through an empty factory. It also runs the simulation once with all randomness off, as a check. Under 5 seconds.

Files. `results/vsm_hand_calc.csv`, `resource_loads.csv`, `deterministic_lead_time.csv`, `deterministic_check.csv`, `docs/process_flow.png`.

Rank	Resource	Load (min/EVA)	Capacity (EVA/week)	Utilisation at 3/week
1	Steeking crew	420.0	5.71	52.5 %
2	Header welder	384.0	6.25	48.3 %
3	Hairpin welder	216.0	11.11	27.0 %
4	Fin press / boxing operator	176.2	13.62	22.0 %
5	Final assembly crew	150.0	16.00	18.8 %

How to read it. The resource with the highest load is the bottleneck: here steeking, at 420 min/EVA. Its capacity is 2,400 / 420 = 5.71 EVA/week, which is the system's maximum output. The critical-path lead time of one order is 10848 min = 22.60 working days. **deterministic_check.csv** must show exact_match = True on every row. That proves the simulation equals the hand calculation.

Step 2 - Analytical equations (fin press, weld stations, Kingman)

```
python -m evapsim analytic
```

What it does. It evaluates the fin-press OEE equations A1-A8 and the metrics for the weld stations, and estimates queue times with the Kingman formula. Under 5 seconds.

Files and meaning. `fin_press_calibration.csv` shows that the calibrated inputs give a job time of 228.6 min per 3 EVA (the whiteboard says 229) and an OEE of 69.99 %. `fin_press_oee_options.csv` shows that SMED alone only lifts OEE to 73.9 %, so reaching 85 % needs a package of changes. `station_metrics.csv` gives, for example, header welding: 8 min/weld, 0.125 welds/min, 48 welds, 384 min/coil, 1.25 EVA per shift.

Step 3 - Input analysis (which probability distribution?)

```
python -m evapsim input
```

What it does. For each operation's 40 time-study readings, it fits three candidate distributions (lognormal, gamma, Weibull). It compares them with the **AIC** score (lower is better) and the **KS test** (p above 0.05 means the fit is not rejected), and selects one by a documented rule. About 10 seconds.

Files. `results/input_analysis.csv` and `results/figures/input_fit_*.png`. In the histogram and Q-Q plots, points on the 45-degree line mean a good fit. **Result:** lognormal for 16 operations and Weibull for steeking. A later test showed the choice does not change any result significantly.

Step 4 - Baseline Monte Carlo (the current state)

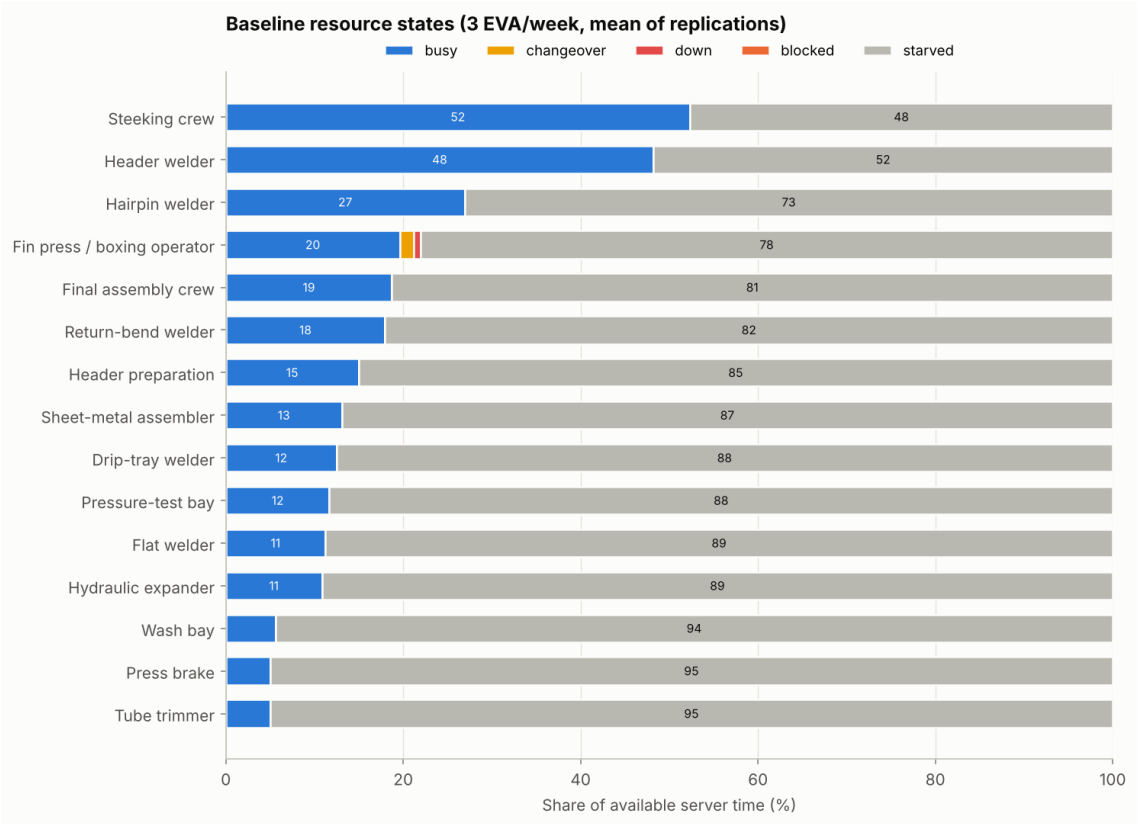
```
python -m evapsim baseline          # or: make baseline (runs steps 1-6)
```

What it does. It simulates the current factory at 3 EVA/week: an 8-week warm-up that is discarded, then 52 measured weeks, repeated 300 times. About 15 seconds.

Metric	Mean	95 % CI half-width	Meaning
Throughput	2.992	±0.013	EVA completed per week
Order lead time, mean	23.123	±0.049	average days from order to last EVA complete
Order lead time, P95	28.242	±0.137	95 % of orders finish faster than this
On-time delivery	0.972	±0.003	share of orders within the 6-week promise
WIP in system	13.655	±0.062	average EVA somewhere in the system
WIP from Little's law	13.649	±0.066	the same, from Little's law (check)
First-pass yield at pressure test	0.960	±0.002	share passing the pressure test first time
Value-added ratio	0.202	±0.000	share of lead time that is actual processing

Files. `baseline_kpis.csv` (the table above), `baseline_replications.csv` (one row per simulated year), `baseline_resources.csv` (states per resource), `baseline_stations.csv` (queue time per station), `baseline_bottleneck_frequency.csv`, `baseline_wip_by_stage.csv`, `convergence*.csv` and figures.

How it fits the study. This is the reference that every scenario is compared with. Note where the average 13.7 EVA in the system sit: 5.3 waiting for purchased material (mostly tubing), 4.6 on the coil line (steeking to test), 1.8 out at coating, 1.0 in preparation, 0.6 in engineering and 0.4 at final assembly. The biggest pile is waiting for suppliers, not in front of the bottleneck. The two WIP estimates (13.65 and 13.65) agree, which confirms Little's law.



baseline_resource_states.png: how each resource spends its time. Blue = busy, yellow = changeover, red = down, grey = idle (starved). The bar at the top is the bottleneck.

Convergence. `convergence_required_replications.csv` says how many replications are needed for a given precision. For a ±1 % CI on mean lead time, 16 replications are needed. With 300 the precision achieved is ±0.21 %.

Step 5 - Validation and calibration

```
python -m evapsim validate
```

What it does. It simulates 300 periods of 26 weeks and compares them with the 26 observed weeks. It reports the % error, the **prediction-interval p-value** (is the observation a plausible outcome of the model? p of 0.05 or more means yes) and a Welch t-test. It also re-runs the calibration grid and the Kingman check. About 2 minutes.

Result. Lead time: observed 24.28 d, simulated 23.14 d, error -4.70 % (target within ±5 %), p = 0.093, verdict valid. With the brief's original 8-day tubing lead time the error was -7.9 %, which fails. `calibration_grid.csv` shows how the smallest adjustment that passes (8.75 days) was chosen. Throughput always matches demand in a stable system, so lead time is the meaningful validation test.

Step 6 - Input uncertainty (two-level Monte Carlo)

```
python -m evapsim uncertainty
```

What it does. It (a) runs 40 plausible steeking times × 10 replications and splits the variance, (b) compares a run with a fixed 420 min steeking time against one with an uncertain steeking time, and (c) runs the stakeholder guesses of 6, 7, 16 and 20 hours. About 1.5 minutes.

Steeking (h/EVA)	Capacity (EVA/wk)	Throughput at 3/wk	Lead time (d)	Stable?
6	6.67	3.00	22.6	yes

Steeking (h/EVA)	Capacity (EVA/wk)	Throughput at 3/wk	Lead time (d)	Stable?
7	5.71	3.00	23.1	yes
16	2.50	2.50	50.8	no: backlog grows
20	2.00	2.01	74.9	no: backlog grows

Meaning. At 5 EVA/week, 86 % of the lead-time variance comes from not knowing the steeking time. The 16-20 h guesses cannot be right, because the line would not even make today's 3 EVA/week, yet it does. Conclusion: do a proper steeking time study before investing.

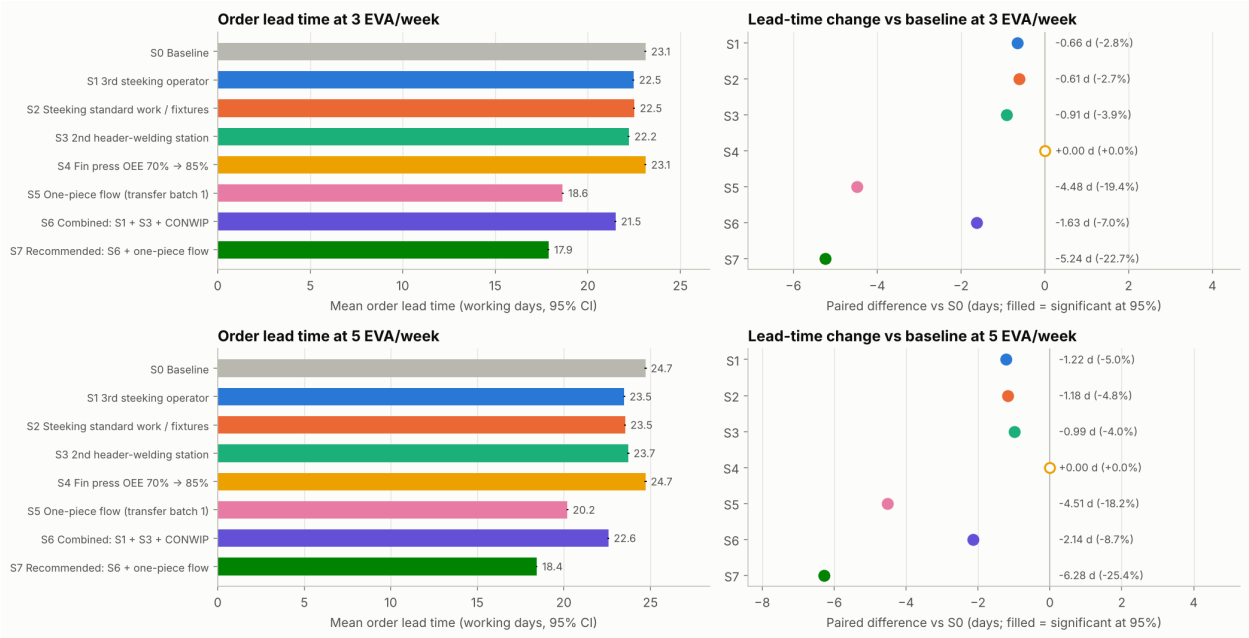
Step 7 - Scenarios S0-S7

```
python -m evapsim scenarios          # or: make scenarios (also draws the VSMs)
```

What it does. It runs all 8 scenarios from `config/scenarios.yaml` at 3 and 5 EVA/week, 300 replications each, with common random numbers. About 4 minutes.

Scenario	LT @3 (d)	Change vs S0 @3	LT @5 (d)	Change vs S0 @5	Bottleneck @5
S0 Baseline	23.1	-	24.7	-	steeking
S1 3rd steeking operator	22.5	-0.66 d (-3 %)	23.5	-1.22 d (-5 %)	header welder
S2 Steeking standard work / fixtures	22.5	-0.61 d (-3 %)	23.5	-1.18 d (-5 %)	header welder
S3 2nd header-welding station	22.2	-0.91 d (-4 %)	23.7	-0.99 d (-4 %)	steeking
S4 Fin press OEE 70% -> 85%	23.1	+0.00 d (+0 %)	24.7	+0.00 d (+0 %)	steeking
S5 One-piece flow (transfer batch 1)	18.6	-4.48 d (-19 %)	20.2	-4.51 d (-18 %)	steeking
S6 Combined: S1 + S3 + CONWIP	21.5	-1.63 d (-7 %)	22.6	-2.14 d (-9 %)	steeking
S7 Recommended: S6 + one-piece flow	17.9	-5.24 d (-23 %)	18.4	-6.28 d (-25 %)	steeking

Files. `scenario_summary.csv` (means and CIs), `scenario_paired_differences.csv` (differences against S0 with CI, p-value and the CRN correlation), `scenario_resources.csv`, `scenario_bottleneck_frequency.csv`, and the figures `scenario_comparison.png` and `scenario_bottlenecks.png`.

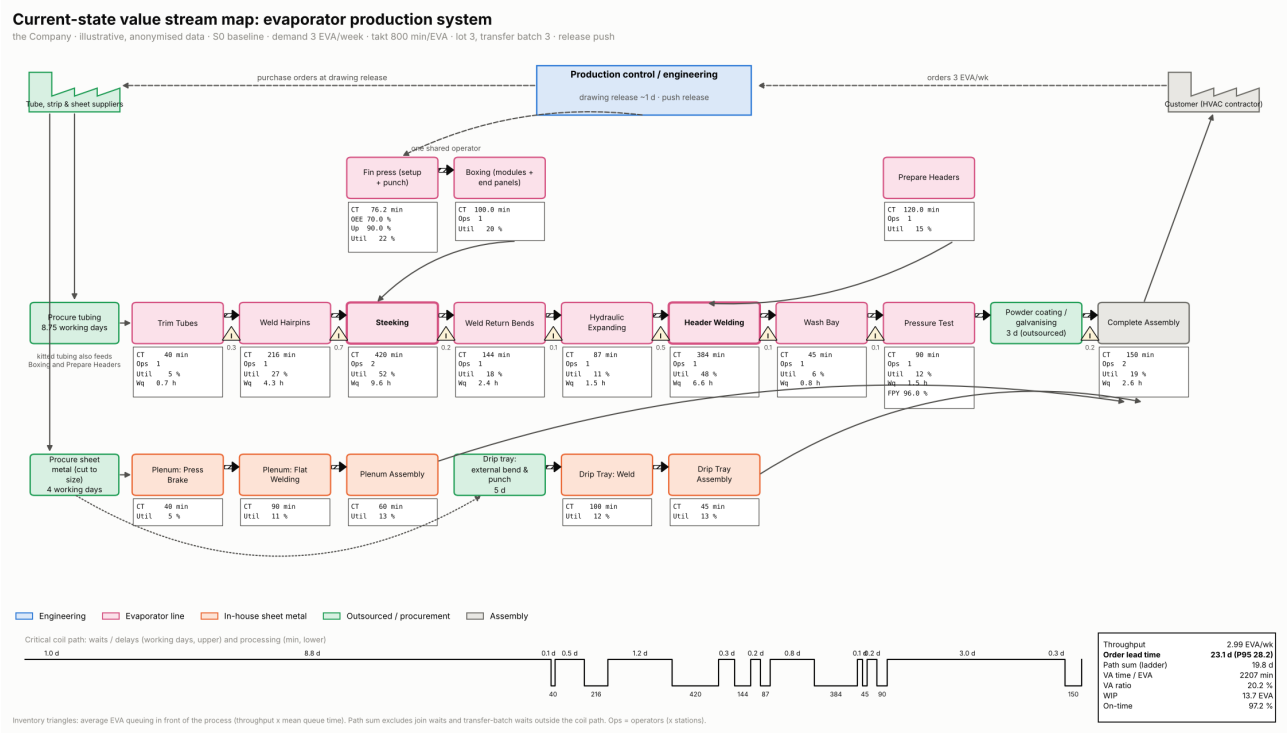


scenario_comparison.png: left, mean lead time per scenario. Right, the paired change against S0 (dot = mean, line = 95 % CI, filled dot = significant).

Step 8 - Value stream maps

```
python -m evapsim vsm
```

What it does. It draws `docs/vsm_current.png` (S0 at 3/week) and `docs/vsm_future.png` (S7 at 5/week). Each process box shows the cycle time, operators, utilisation and queue time. The yellow triangles show the average number of EVA waiting. The ladder at the bottom shows waits (upper steps, in days) against processing (lower steps, in minutes) along the critical path. About 30 seconds.

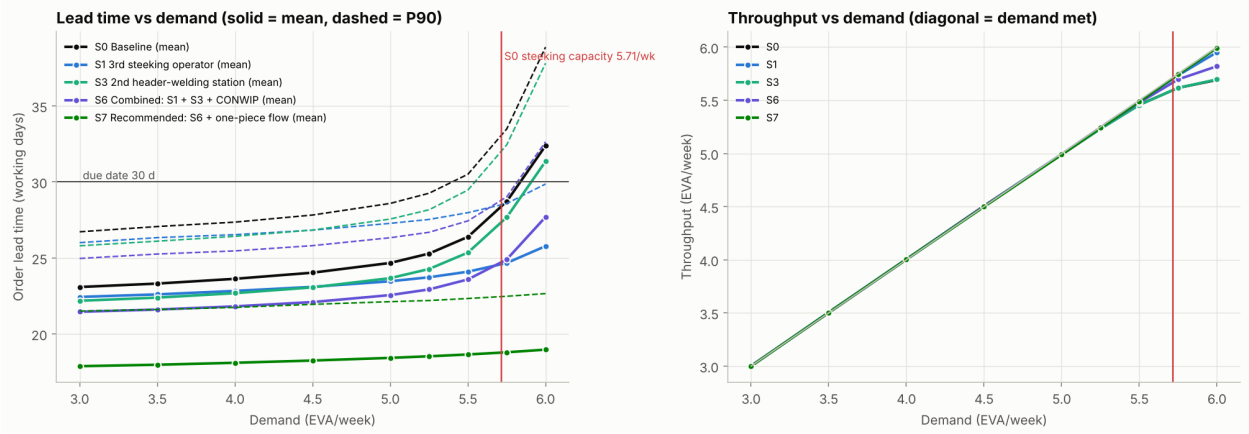


vsm_current.png: the current-state VSM drawn from the simulation.

Step 9 - Demand sweep (the hockey stick)

```
python -m evapsim sweep
```

What it does. It runs demand from 3 to 6 EVA/week for S0, S1, S3, S6 and S7. About 3 minutes. **File:** `demand_sweep.csv`, `demand_curve.png`.



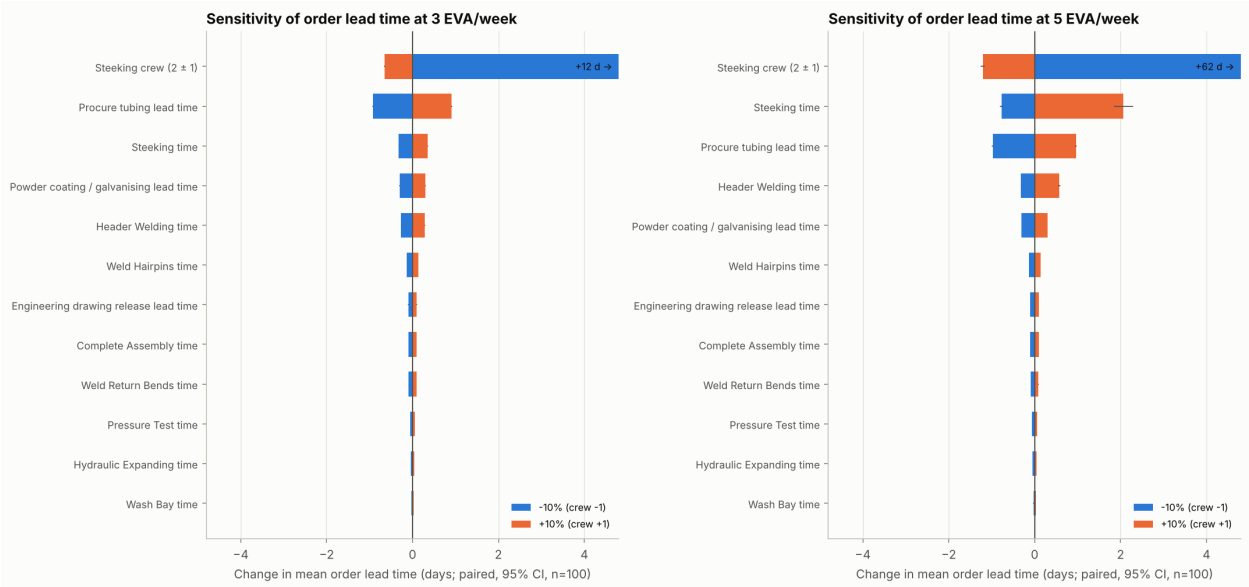
demand_curve.png: lead time stays flat until demand approaches the bottleneck capacity (red line), then rises sharply. On the right, throughput below the diagonal means demand is no longer met.

How to read it. The knee of the curve is the practical capacity. Plan to operate below it. Where a curve stays flat (S7), that package has enough capacity for the demand.

Step 10 - Sensitivity (tornado chart)

```
python -m evapsim sensitivity
```

What it does. It changes one input at a time by ±10 % (the steeking crew by ±1 person) and measures how much lead time moves. About 4 minutes. **Files:** `sensitivity_oat.csv`, `sensitivity_tornado.png`, `distribution_robustness.csv`.



sensitivity_tornado.png: the longer the bar, the more that input matters. Removing a steeker makes the line unstable (the bar runs off the chart).

How it fits. It tells you which inputs deserve better data (steeking time, tubing lead time) and which hardly matter (wash time, expansion).

Step 11 - Update the documentation tables

```
python -m evapsim report
```

This rewrites every results table in README.md and docs/*.md from the CSV files, so the documents always match the latest run. Run it after any change.

Step 12 - The analysis notebook

```
python -m jupyter notebook notebooks/analysis.ipynb
# or re-run it non-interactively:
make notebook
```

A browser tab opens. Use Run, Run All Cells to walk through the study with tables, charts and live calculations.

Step 13 - The interactive dashboard

```
streamlit run app/dashboard.py          # or: make dashboard
```

A browser tab opens at http://localhost:8501. Press **Ctrl + C** in the terminal to stop it. Section 9 explains how to use it.

7. What every metric means

Metric (column name)	Definition	Unit	What good looks like
Throughput (throughput_eva_week)	EVA finished per measured week	EVA/week	Equal to demand. Lower means the system is overloaded.
Demand attainment (attainment)	Throughput ÷ demand	ratio	1.0
Weekly compliance (weekly_compliance)	Share of weeks with output at or above demand. It is noisy because orders come in lots of 3.	share	High, but read it with attainment
Order lead time (lt_mean_days)	Time from order arrival to completion of its last EVA	working days	Short and stable
P50 / P90 / P95 lead time	The time within which 50 / 90 / 95 % of orders finish	working days	P95 below the promise (30 d)
On-time delivery (on_time)	Share of orders finished within 6 weeks of arrival	share	95 % or more
WIP (wip_total, wip_)	Average number of EVA in the system, or in each stage	EVA	Low, and not piling up in front of the bottleneck
Little's-law WIP (littles_wip)	Completion rate × average EVA lead time	EVA	Equal to wip_total (a correctness check)
Utilisation (busy)	Share of available time a resource is working	share	Bottleneck at or below about 85 % for stable lead times
Active share (active)	Busy + changeover + down. Used to find the bottleneck	share	-
Starved / changeover / down / blocked	Idle; setting up; broken down; finished but unable to pass work on (always 0 here, because queues are unlimited)	share	-
Queue time (mean_wait_min)	Average time waiting before a station, including waiting for the other EVA in the lot	min	Short at the bottleneck
Bottleneck frequency	Share of simulated years in which a resource has the highest active share	share	100 % at one resource = a clear bottleneck
First-pass yield (fpy_test)	Share passing the pressure test first time	share	About 96 % (input: 4 % failure)
Value-added time / ratio	Processing minutes per EVA, and that ÷ EVA lead time	min, share	A higher ratio means less waiting
OEE (fin press)	Availability × performance × quality	share	70 % now
Paired difference (diff_mean, CI, p_value)	Scenario minus S0 over the same random years. Significant if the CI excludes 0	same as metric	CI fully below 0 for lead time
crn_corr	Correlation between S0 and the scenario across replications	-	High (CRN is working)
Variance shares (share_input / share_process)	Part of the variance caused by not knowing the steeking time, and part caused by everyday variability	share	A high input share means you need a time study
Validation error / PI p-value	Simulated vs observed mean; plausibility of the observation	%, -	Error within ±5 % and p of 0.05 or more

Working days and minutes. The model clock counts working minutes only: 480 per day and 5 days per week, so 2,400 per week. Lead times are reported in working days (= minutes ÷ 480).

8. Editing the model parameters

8.1 Where and how

1. Open `config/line_parameters.yaml` in a text editor (VS Code, Notepad, TextEdit in plain-text mode). Do **not** use Word.
2. Change the number after the colon. Keep the indentation exactly as it is, and use spaces, **never tabs**. Text after # is a comment.
3. Save the file.
4. Check that the model accepts it: `python -m pytest`. A wrong value gives a clear ConfigError message listing what is wrong.
5. Re-run the affected steps (usually `python -m evapsim baseline` and `scenarios`), then `python -m evapsim report`.

> Some tests compare with the original whiteboard numbers. After deliberate input changes, a few of those tests can fail (for example the exact 52.5 % steeking utilisation). That is expected. Use `python -m evapsim handcalc` and check that `deterministic_check.csv` is still all True: that is the check that matters.

8.2 The parameters you are most likely to change, and why

Parameter (in line_parameters.yaml)	Example	Why you would change it
operations.steeking.mean_min	420 → 455	After a real time study of the bottleneck. This is the most important input.
operations..mean_min or t_op_min / n_ops	header_welding t_op_min: 8 → 7	New standard times, a new fixture, a new weld process.
operations..cv	0.20 → 0.10	Variability observed in the time study (standard work reduces it).
resources.steeking_crew.crew	2 → 3	Test adding an operator. Elapsed time = labour ÷ (crew × efficiency).
resources.header_welder.servers	1 → 2	Test a second station (a parallel machine or welder).
demand.eva_per_week	3 → 5	Plan for growth. Watch the bottleneck capacity of 5.71.
demand.due_date_weeks	6 → 5	A different delivery promise changes on-time %.
orders.transfer_batch	3 → 1	One-piece flow: EVA move individually.
orders.lot_size	3 → 2	Smaller work orders.
release.rule / conwip_limit_orders	push → conwip, 3	Limit how many orders are on the shop floor at once.
lead_times.procurement_tubing.mean_days	8.75 → 6	Negotiate a supplier lead time. It is the biggest non-capacity lever.
fin_press.setup_min, mtbf_min, mttr_min, performance, quality	30 → 18	SMED/TPM studies on the fin press.
rework.fail_prob	0.04 → 0.02	A quality improvement at the pressure test.
calendar.minutes_per_day	480 → 960	Add a second shift (with productive minutes).
time_scale_factor	1.0 → 0.8	Scale all process times at once, for anonymising data or quick what-ifs.
simulation.replications	300 → 100	Faster runs; wider CIs.
simulation.deterministic	false → true	Verification mode: no randomness. The run reproduces the hand calculation.
flags.wash_before_test	true → false	Test the flowchart's original order.

8.3 Worked example: you measured steeking at 7.6 hours

1. Open the YAML and find `steeking:` under `operations:`.
2. Set `mean_min:` 456 (7.6 × 60) and `source: "time study"`, and set `cv` to the CV you measured.
3. Run `python -m evapsim handcalc`. The new capacity is 2,400 / 456 = 5.26 EVA/week.
4. Run `python -m evapsim baseline` and `validate`. Check that the validation verdict is still valid.
5. Run `scenarios`, `sweep` and `report`, and read the new knee of the demand curve.

8.4 Defining a new scenario

Scenarios live in `config/scenarios.yaml`. A scenario lists only what changes compared with the baseline. `{scale: 0.9}` multiplies the current value and `{add: -2}` adds to it. `extends` builds on other scenarios.

```
S8:
  name: "Faster tubing supplier"
  description: "Tubing lead time cut by 3 days, plus one-piece flow."
  extends: [S5]
  overrides:
    lead_times:
      procurement_tubing: {mean_days: {add: -3}}
```

Save it and run `python -m evapsim scenarios`. S8 appears in all scenario tables and charts, compared fairly against S0.

8.5 Replacing the illustrative data with real data

1. Time study: put at least 30 readings per operation into `data/time_study_samples.csv` (same columns), then run `python -m evapsim input`. Copy each fitted mean and CV into the YAML, and set `source: "time study"`.
2. Production records: replace `data/observed_weekly_output.csv` with your weekly output and order lead times (in working days), for at least 20 weeks.
3. Run `python -m evapsim validate`. If the error is outside ±5 %, read `calibration_grid.csv` and adjust the most uncertain assumed input (usually a supplier lead time). Never adjust a value you measured just to pass validation.
4. Record what you changed and why in `docs/ASSUMPTIONS.md`.

9. Using the dashboard

1. Run `streamlit run app/dashboard.py`. Your browser opens the dashboard.
2. **Sidebar 1, Starting point:** pick a scenario (S0-S7) as the base.
3. **Sidebar 2, Key inputs:** demand, transfer batch, steeking crew, header-welding stations, release rule and CONWIP limit, time scale, and the steeking-uncertainty toggle.
4. **Main table:** double-click a cell to edit an operation time or CV.
5. **Sidebar 3, Run settings:** 30 replications is a good balance (about 5-15 s). Tick Compare with S0 to see paired differences.
6. Press **Run simulation**. The KPI tiles show the result and, underneath, the change against S0. Hover over the ? icon for the 95 % CI.
7. Tabs: Lead time (distribution, with the due date in red), Resource states, Bottleneck, Demand curve (your run is the red star) and Scenario comparison.

> A red banner warning that steeking capacity is below demand means the factory cannot keep up. Lead time then keeps growing the longer you simulate.

10. Troubleshooting

Problem	Cause	Fix
'python' is not recognized	Python not on PATH	Reinstall Python and tick "Add to PATH", or use py on Windows
running scripts is disabled	PowerShell policy	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
No module named evapsim / simpy	Virtual environment not active, or not installed	Activate .venv (5.5), then pip install -r requirements.txt and pip install -e .
'make' is not recognized	Windows has no make	Use the python -m evapsim commands
ConfigError: Invalid configuration	A bad value in the YAML	Read the listed lines and fix the values or the indentation
Results differ from the README	Inputs, seed or replications changed	Expected after edits. Run python -m evapsim report
Lead time huge, throughput below demand	Demand above bottleneck capacity	Lower demand or add capacity. Check resource_loads.csv
Dashboard stuck on Running	Process pool problem	Set Worker processes to 1
Very slow	Many replications or a long horizon	Use --quick or fewer replications

11. Quick reference card

```
# every session
cd evaporator-vsm-simulation
source .venv/bin/activate            # Windows: .venv\Scripts\Activate.ps1

# check
python -m pytest                     # 48 passed

# the study, in order
python -m evapsim handcalc           # 1 hand-calculated VSM + exact check
python -m evapsim analytic           # 2 fin press OEE, weld metrics, Kingman
python -m evapsim input              # 3 distribution fitting
python -m evapsim baseline           # 4 current state, 300 replications
python -m evapsim validate           # 5 validation + calibration
python -m evapsim uncertainty        # 6 two-level Monte Carlo
python -m evapsim scenarios          # 7 S0-S7 at 3 and 5 EVA/week
python -m evapsim vsm                # 8 current/future value stream maps
python -m evapsim sweep              # 9 demand curve (hockey stick)
python -m evapsim sensitivity        # 10 tornado chart
python -m evapsim report             # 11 refresh README/docs tables
make notebook                        # 12 re-run the analysis notebook
streamlit run app/dashboard.py       # 13 interactive dashboard

# all at once
make all        # about 16 min       make quick   # 30 replications, about 5 min
python -m evapsim baseline --quick   # any step in quick mode -> results/quick/
```

Further reading in the repository: `README.md` (results and equations), `docs/OPERATING_PROCEDURES.md` (SOP with checklists), `docs/ASSUMPTIONS.md` (every assumption and data source), `docs/MODEL_DESCRIPTION.md` (technical model), `docs/BUILD_PHASES.md` (how it was built).